SECOND EDITION

A PRACTICAL GUIDE TO BUILDING

# Software That Can Change

Software Design Principles, Trade-offs, and Architecture for Working Developers



MOISES GAMIO

# A Practical Guide to Building Software That Can Change

## Software Design Principles, Trade-offs, and Architecture for Working Developers

Moises Gamio

Second Edition

*A Practical Guide to Building Software That Can Change*
Second Edition

Copyright © 2022, 2026 Moises Gamio
All rights reserved.

The first edition was published in 2022 as *Software Design Principles: A Practical Guide*.

ISBN 9798161952931 (paperback)

Companion code: github.com/mgamio/subFlow
codersite.dev

# Contents

**Preface** ..... ix

**Introduction: Why Design Matters** ..... xiii

**How to Use This Book** ..... xvii

PART I · CHANGEABLE CODE

**1. Core Design Principles** ..... 3

Don't Repeat Yourself (DRY) ..... 4
Keep It Simple, Stupid (KISS) ..... 13
You Aren't Gonna Need It (YAGNI) ..... 18
Loose Coupling and High Cohesion ..... 23
Separation of Concerns ..... 30
Clean Code ..... 36

**2. Object-Oriented Design: Protecting Your Rules** ..... 49

Classes and Objects: A Quick Refresher ..... 50
Encapsulation and Invariants ..... 52
Inheritance and Its Limits ..... 59
Polymorphism ..... 68
Abstraction ..... 72
Relationships Among Classes ..... 79

**3. SOLID Principles** ..... 83

Single Responsibility Principle (SRP) ..... 84
Open-Closed Principle (OCP) ..... 89
Liskov Substitution Principle (LSP) ..... 97
Interface Segregation Principle (ISP) ..... 103
Dependency Inversion Principle (DIP) ..... 110

**4. Design for Testability** ..... 121

What Tests Are For ..... 122
Testing Failure Cases ..... 124
Deterministic Code ..... 131
Test Behavior, Not Implementation ..... 135
Contract Tests ..... 139

Property-Based and Mutation Testing ..... 142
Testing Legacy Code ..... 146
Tests Reveal Your Boundaries ..... 151

**5. When Principles Conflict ..... 155**
Principles Are Heuristics ..... 156
The Decision Questions ..... 156
Eight Common Conflicts ..... 158
Recording the Decision ..... 169
A One-Page Decision Guide ..... 171

PART II · CHANGEABLE SYSTEMS

**6. Designing Boundaries: Domain, Data and APIs ..... 177**
What Makes a Boundary Good ..... 178
Entities, Value Objects and Invariants ..... 179
API Contracts and Errors ..... 181
Idempotent APIs and Duplicate Messages ..... 184
Pagination and Filtering ..... 187
Compatibility and Versioning ..... 189
Authorization at the Boundary ..... 191
Transactions, Concurrency and Events ..... 192
Data That Lasts: Constraints and Migrations ..... 196
Caching ..... 199

**7. From Requirements to Architecture ..... 203**
Functional and Quality Requirements ..... 204
A Repeatable Process ..... 205
Service Boundaries and Data Ownership ..... 218
Communicating the Architecture ..... 219

**8. Communicating Design: UML and Beyond ..... 223**
The Design Problem ..... 224
The Principle: Draw to Answer a Question ..... 225
Which Diagrams Pay Off ..... 226
Class Diagrams ..... 227
Diagrams of Behavior ..... 232
Diagrams of the System: the C4 Model ..... 235
Diagrams of Data ..... 236
Keeping Diagrams True ..... 238

UML and Its Neighbors ..... 241
Benefits, Costs and When Not to Draw ..... 242

PART III · DESIGN IN PRACTICE

**9. Case Studies: Three Systems from a Blank Page ..... 249**
A Repeatable Walk-Through ..... 250
Case Study 1: Reading a Functional Description ..... 250
Case Study 2: Designing the Ordering API ..... 257
Case Study 3: A B2B Product Integration ..... 268
The Principles at Work ..... 283
Benefits, Costs and When Not to Use the Walk-Through ..... 284

**10. Soft Skills for Design Decisions ..... 287**
The Design Problem: A Good Decision That Almost Lost ..... 288
Skill 1: Speak in the Listener's Units ..... 290
Skill 2: Bring Options, Not a Verdict ..... 294
Skill 3: Write One Page Before the Meeting ..... 296
Skill 4: Review the Design, Not the Designer ..... 298
Skill 5: Turn Disagreement into a Decision ..... 304
Benefits, Costs and When Not to Apply These Skills ..... 307

**11. Conclusions: Growing as a Designer ..... 311**
SubFlow's Journey on One Page ..... 312
Six Ideas That Kept Returning ..... 314
Improving Your Abstraction Ability ..... 315
A Growth Path ..... 318
Your Next Three Months ..... 320
A Last Exercise: Family Plans ..... 321
Final Words ..... 321

APPENDICES

**A. All Checklists in One Place ..... 323**

**B. From Symptom to Principle ..... 335**

**C. Glossary ..... 341**

**D. Further Reading ..... 347**

**About the Author ..... 351**

CHAPTER 1

# Core Design Principles

*"It should have been a one-line change."*

Every developer has heard this sentence in a stand-up meeting. The product owner wants the annual discount lowered from 20% to 15%. You find the number, change it, run the tests and deploy. Two days later, the support team forwards a dozen emails from angry customers: the checkout page promised one price, and the invoice charged another.

Nothing was wrong with the syntax. The code compiled, the tests passed and every method did what its name said. The problem was the *design*: the same business rule lived in two places, and only one of them received the change.

This chapter is about preventing that kind of failure. Software design principles, such as DRY, KISS, YAGNI, loose coupling and high cohesion, separation of concerns and clean code, capture the experience of developers who learned these lessons the hard way. They give you a vocabulary to spot problems early and a set of moves to fix them.

They are not laws. Each principle solves a specific problem, and each one has a cost. Applied mechanically, a principle can make code worse: DRY can tie together things that should evolve separately, and YAGNI can become an excuse to ignore a requirement everybody knows is coming. So for every principle, you will learn not only *how* to apply it but also *when not to*.

Nor do you need all of them on the first day. Design is iterative. Start with the simplest code that works, and introduce a principle when the code gives you a reason to. If you have kept the code clean along the way, each change will be small.

### A system that grows with the book

Instead of isolated toy examples, this chapter follows one system: **SubFlow**, a small subscription-billing service for a software company. Customers choose a plan (Basic or Pro), pay monthly or annually, receive invoices and can cancel within a refund window.

SubFlow starts simple and a little messy, the way real projects usually start. As the book progresses, it gains new payment providers, notifications, reporting, persistence and external integrations, and each new requirement puts pressure on its design. You will see every principle applied to the same evolving code, which is exactly how you will meet these principles at work.

### How each principle is presented

Every section in this chapter follows the same path, so you always know where you are:

1. **The design problem**: what goes wrong in real projects.
2. **A problematic example**: code that works today but hurts tomorrow.
3. **Symptoms**: how to recognize the problem in your own codebase.
4. **The principle**: what it really says.
5. **The refactoring**: step by step.
6. **Benefits**, followed by **costs and trade-offs**.
7. **When not to apply it**.
8. **Exercises** and a short **checklist** you can use in code reviews.

## Don’t Repeat Yourself (DRY)

### The design problem

Business rules change. Prices, discounts, tax rates, validation rules and status workflows change far more often than the code around them. When a rule is written in several places, every change becomes a search mission, and the place you miss becomes a bug in production.

## A problematic example

In the first version of SubFlow, two developers worked in parallel. One built the checkout page; the other built the monthly invoicing job. Both needed the subscription price.

```
public class CheckoutService {

  public BigDecimal priceFor(String plan, boolean annual) {
    BigDecimal monthly = switch (plan) {
      case "BASIC" -> new BigDecimal("9.99");
      case "PRO"   -> new BigDecimal("19.99");
      default -> throw new IllegalArgumentException(
          "Unknown plan: " + plan);
    };
    if (annual) {
      // 12 months with a 20% discount
      return monthly.multiply(BigDecimal.valueOf(12))
          .multiply(new BigDecimal("0.80"))
          .setScale(2, RoundingMode.HALF_UP);
    }
    return monthly;
  }
}
public class InvoiceService {

  public BigDecimal amountDue(Subscription subscription) {
    BigDecimal monthly = subscription.getPlan().equals("PRO")
        ? new BigDecimal("19.99")
        : new BigDecimal("9.99");
    if (subscription.isAnnual()) {
      return monthly.multiply(new BigDecimal("12"))
          .multiply(new BigDecimal("0.8"))
          .setScale(2, RoundingMode.HALF_UP);
    }
    return monthly;
  }
}
```

Look closely, and you will notice three things:

- The pricing rule (monthly price, twelve months, annual discount) appears in both classes.
- The two copies do not even look alike. One uses a `switch`, the other a conditional operator. One writes the discount as `"0.80"`, the other as `"0.8"`. A text search for `0.80` finds only one of them.

- The copies have *already* drifted apart. `CheckoutService` rejects an unknown plan, while `InvoiceService` silently bills it as Basic.

Now marketing lowers the annual discount to 15%. The developer on duty finds the number in `InvoiceService`, changes it and closes the ticket. From that moment, a customer who buys the Pro plan annually sees **$191.90** at checkout and receives an invoice for **$203.90**.

## Symptoms

You can recognize this problem in your own codebase when:

- A change to one business rule requires edits in several files.
- The same literal (`0.80`, `"PRO"`, `12`) appears in different classes.
- Bug reports say "screen A shows X, but report B shows Y."
- Someone asks during a code review: "Is this calculated anywhere else?"
- A bug you fixed months ago reappears, because it lived in a second copy.

## The principle

Andrew Hunt and David Thomas introduced DRY in *The Pragmatic Programmer* (1999):

> *"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system."*

The key word is **knowledge**, not *code*. DRY is about business rules, decisions and facts. It is not about lines of text that happen to look the same. Two snippets with different syntax can violate DRY, as in SubFlow. Two identical snippets can be perfectly fine, as you will see in *When not to apply it*.

## The refactoring

### Step 1: Name the knowledge

What exactly is duplicated? Two facts: the monthly price of each plan, and the rule for annual billing. Naming them tells you where they belong.

### Step 2: Give each fact one home

The monthly price is a property of the plan, so it lives with the plan. The billing cycle becomes a type instead of a `boolean`.

```
public enum Plan {
  BASIC(new BigDecimal("9.99")),
  PRO(new BigDecimal("19.99"));

  private final BigDecimal monthlyPrice;

  Plan(BigDecimal monthlyPrice) {
    this.monthlyPrice = monthlyPrice;
  }

  public BigDecimal monthlyPrice() {
    return monthlyPrice;
  }
}

public enum BillingCycle { MONTHLY, ANNUAL }
```

### Step 3: Put the rule in one place

```
public class PricingPolicy {

  private static final BigDecimal ANNUAL_DISCOUNT =
      new BigDecimal("0.15");
  private static final int MONTHS_PER_YEAR = 12;

  public BigDecimal priceFor(Plan plan, BillingCycle cycle) {
    if (cycle == BillingCycle.MONTHLY) {
      return plan.monthlyPrice();
    }
    BigDecimal fullYear = plan.monthlyPrice()
        .multiply(BigDecimal.valueOf(MONTHS_PER_YEAR));
    return fullYear
        .multiply(BigDecimal.ONE.subtract(ANNUAL_DISCOUNT))
        .setScale(2, RoundingMode.HALF_UP);
  }
}
```

### Step 4: Make every caller use it

The `Subscription` now stores a `Plan` and a `BillingCycle` instead of strings. You will see the full class in the next section.

```
public class CheckoutService {
  private final PricingPolicy pricing;

  public CheckoutService(PricingPolicy pricing) {
    this.pricing = pricing;
  }

  public BigDecimal priceFor(Plan plan, BillingCycle cycle) {
    return pricing.priceFor(plan, cycle);
  }
}

public class InvoiceService {
  private final PricingPolicy pricing;

  public InvoiceService(PricingPolicy pricing) {
    this.pricing = pricing;
  }

  public BigDecimal amountDue(Subscription subscription) {
    return pricing.priceFor(subscription.plan(),
                            subscription.cycle());
  }
}
```

Now the discount change really is a one-line change: `ANNUAL_DISCOUNT` in `PricingPolicy`. Checkout and invoicing can no longer disagree, because neither of them has its own opinion about prices anymore. And because plans are now an `enum` instead of a `String`, an unknown plan cannot even reach the pricing code.

Figure 1.1 summarizes the refactoring.

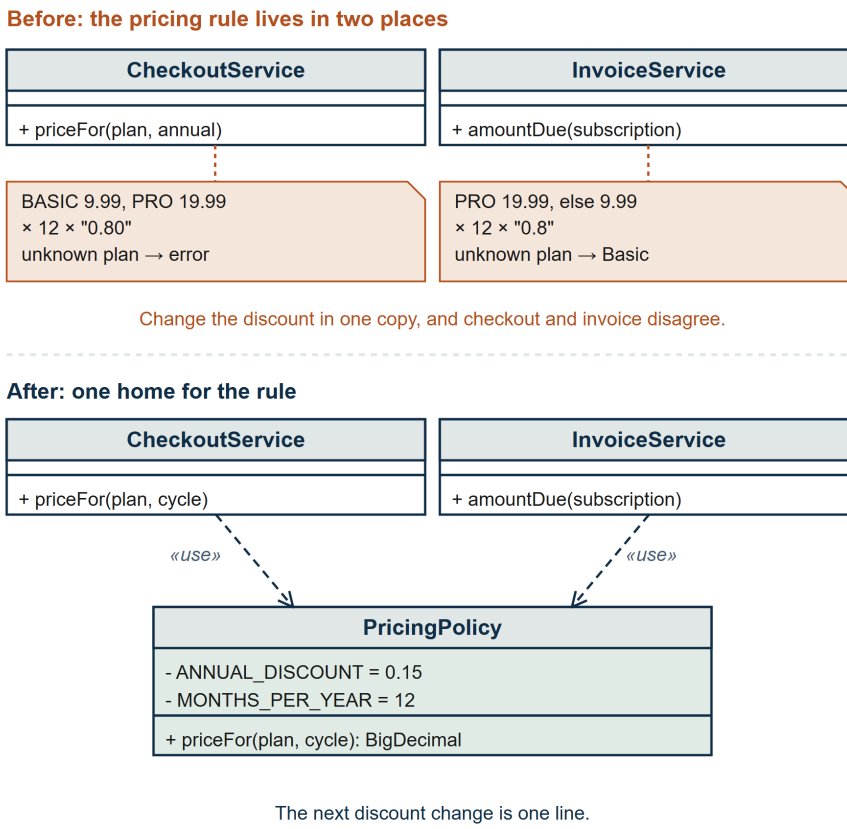



Figure 1.1 – One home for the pricing rule

Notice that both services receive `PricingPolicy` through their constructors instead of creating it themselves. This small decision will matter in the chapter on SOLID, when SubFlow needs a different pricing policy for partner companies.

## Benefits

- **One change, one place.** The next price change takes minutes, not a search across the codebase.
- **Consistency.** Every feature that shows or charges a price agrees, by construction.
- **Discoverability.** A new team member looking for "how we price subscriptions" finds a class with exactly that name.

- **Focused tests.** You test the rule once, thoroughly, instead of testing every copy:

```
class PricingPolicyTest {
  private final PricingPolicy pricing = new PricingPolicy();

  @Test
  void annualPlanGetsTheDiscount() {
    assertEquals(new BigDecimal("203.90"),
        pricing.priceFor(Plan.PRO, BillingCycle.ANNUAL));
  }

  @Test
  void monthlyPlanPaysTheListPrice() {
    assertEquals(new BigDecimal("19.99"),
        pricing.priceFor(Plan.PRO, BillingCycle.MONTHLY));
  }
}
```

## Costs and trade-offs

Removing duplication is not free. Before you extract shared code, weigh these costs:

- **Coupling.** Every class that uses `PricingPolicy` now depends on it. For a real business rule, that is exactly what you want. For code that only *looks* shared, it ties together parts of the system that should change independently.
- **Indirection.** The reader must jump to another class to understand the price. A good name keeps this cost low; a vague name such as `Utils` or `Helper` makes it high.
- **The wrong abstraction.** Sandi Metz put it memorably: "duplication is far cheaper than the wrong abstraction." Watch for a shared method that keeps growing flags for each caller, such as `priceFor(plan, cycle, isPartner, isLegacy, skipDiscount)`. It no longer represents one piece of knowledge; it hides several behind one name.
- **Team and service boundaries.** A shared library between two independently deployed services couples their release schedules. Duplicating a small amount of code across a service boundary is often the better trade-off.

### When not to apply it

The most common DRY mistake is merging code that is only **coincidentally similar**. Consider two rules in SubFlow:

```
public static final int FREE_TRIAL_DAYS = 14;
public static final int REFUND_WINDOW_DAYS = 14;
```

A well-meaning refactoring replaces both with a single `DEFAULT_PERIOD_DAYS = 14`. Next quarter, marketing runs a promotion that extends the free trial to 30 days. With the merged constant, the refund window silently becomes 30 days too: a financial decision nobody approved.

The two numbers are equal today, but they represent different knowledge, owned by different people, changing for different reasons. Ask one question:

> **If one of them changes, must the other change too, for the same reason?** If the answer is no, it is coincidence, not duplication. Leave it alone.

Other situations where repetition is acceptable:

- **Test code.** A test that reads clearly from top to bottom is worth more than a test that saves three lines through a shared helper. Readable tests often repeat their setup on purpose.
- **Similar code with an unclear relationship.** Many developers follow the *rule of three*: wait for the third occurrence before extracting, because by then you understand what the copies really have in common. When you already know two copies are the same business rule, as with SubFlow's pricing, two copies are already one too many.

### DRY in the small

The same idea applies to small pieces of knowledge inside a single class. String literals that represent a fixed set of values belong in an `enum`:

```
// Before: the valid statuses exist only in developers' heads
if (status.equals("ACTIVE")) { ... }
else if (status.equals("CANCELED")) { ... }

// After: one authoritative list, checked by the compiler
public enum SubscriptionStatus { TRIAL, ACTIVE, CANCELED }
```

And when the authoritative representation already exists, use it instead of creating your own. Do not declare `PI = 3.14159`; the JDK already provides `Math.PI`.

### Exercises

1. Pick one business rule in your current project, such as a tax rate, a discount or a status transition. In how many places is it implemented? Do all of them agree?
2. SubFlow adds a Team plan at $49.99 per month, with the same annual discount. List the files you would change *before* the refactoring and *after* it.
3. A colleague proposes merging `formatInvoiceDate()` and `formatLogTimestamp()` because they produce the same output today. Should you? Which question decides it?
4. Find a shared method with two or more `boolean` parameters in your codebase. Does it represent one piece of knowledge or several?

### Checklist

☐ Is this the same *knowledge*, or just similar text?

☐ Would both copies change for the same reason, at the same time?

☐ Does the rule have one obvious home with a meaningful name?

☐ Is the shared code growing flags to behave differently for each caller?

☐ Does sharing this code couple teams or services that release independently?

# Keep It Simple, Stupid (KISS)

## The design problem

Most code is read many more times than it is written. Every extra branch, flag variable or clever trick is a small tax that every future reader pays: the colleague fixing a bug at midnight, the new team member, and you, six months from now. Complex code also hides bugs, because nobody can hold all of its paths in their head at once.

## A problematic example

SubFlow needs to know whether a subscription is active on a given day, to decide whether a customer can log in. Here is the first version, written under time pressure and patched twice since:

```
public boolean isActive(Subscription subscription,
                        LocalDate today) {
  boolean active = false;
  if (subscription.status() != SubscriptionStatus.CANCELED) {
    if (subscription.endDate() == null) {
      active = true;
    } else {
      if (subscription.endDate().isAfter(today)) {
        active = true;
      } else if (subscription.endDate().isEqual(today)) {
        active = true;
      } else {
        active = false;
      }
    }
  } else {
    active = false;
  }
  return active;
}
```

The code works. But to answer one simple question, it uses a flag variable, four levels of nesting and three separate assignments of `false`, two of them unnecessary. The real rule is hidden in the noise.

## Symptoms

- You need to trace the code with pen and paper to know what it returns.
- Deep nesting: the code drifts to the right of the screen.

- Flag variables that are assigned in many branches and returned at the end.
- Conditions that check something already known (`x > 50 && x <= 100` right after `if (x > 100)`).
- Code reviewers say "I think this is right" instead of "this is right."

## The principle

The KISS principle is usually attributed to Kelly Johnson, an aircraft engineer at Lockheed, who wanted jet aircraft that an average mechanic could repair in the field with basic tools. Translated to software: **prefer the solution that the next developer can understand and change safely**.

Simple does not mean short, and it does not mean "no abstractions." It means that the structure of the code matches the structure of the problem, without extra parts. Extreme Programming phrases it as a question: *what is the simplest thing that could possibly work?*

## The refactoring

### Step 1: Pin down the current behavior

Before simplifying logic, protect yourself with tests that describe what the code does today: a canceled subscription, one with no end date, one ending today, one that ended yesterday. If the simplified version passes the same tests, you have not changed the behavior.

### Step 2: Return early instead of tracking a flag

As soon as you know the answer, return it. The canceled case is decided in the first line.

### Step 3: Say the rule in one expression

"Ends today or later" is the same as "does not end before today." The `java.time` API lets you say it directly.

### Step 4: Put the logic next to the data

The rule only uses the subscription's own fields, so it belongs in `Subscription`. Here is the full class, as a Java 17 record:

```
public record Subscription(
    long id,
    long customerId,
    Plan plan,
    BillingCycle cycle,
    SubscriptionStatus status,
    LocalDate startDate,
    LocalDate endDate) {     // null = renews automatically

  public boolean isActiveOn(LocalDate day) {
    if (status == SubscriptionStatus.CANCELED) {
      return false;
    }
    return endDate == null || !endDate.isBefore(day);
  }
}
```

The rule now reads like the business explains it: *a subscription is active unless it was canceled or it has already ended.* Callers write `subscription.isActiveOn(today)`.

## Benefits

- **Fewer places for bugs to hide.** Two conditions instead of six branches.
- **Faster reviews.** A reviewer can verify the rule at a glance.
- **Easier change.** When the business adds a grace period, there is one obvious line to change.

## Costs and trade-offs

- **Simple is harder to write than complex.** Finding the simple version takes time and understanding. The first draft is rarely the simple one; the simple one usually appears in the second or third pass.
- **Simple for whom?** A stream pipeline is simple for a team that uses streams every day and confusing for a team that does not. Agree on your team's idioms.

- **Compact is not the same as simple.** A one-line expression with three nested conditional operators is short, but it is not simple. You will see an example in the Clean Code section.

### When not to apply it

KISS is not permission to ignore real complexity. The danger is the **simplistic** solution, one that is easy to write because it ignores part of the problem:

```
double total = 0.1 + 0.2;
System.out.println(total);   // 0.30000000000000004
```

Using `double` for money looks simpler than `BigDecimal`, but it produces wrong invoices. The same is true for writing your own date arithmetic instead of using `java.time`, or ignoring time zones in a system with customers on three continents. When the problem is genuinely complex, the simple solution is the one that handles that complexity in a clear, well-named place, as `PricingPolicy` does.

Also notice that KISS did not stop us from adding a class in the DRY section. `PricingPolicy` added a new type, yet the system became simpler, because the rule now has one clear home. KISS is about reducing what the reader must understand, not about minimizing the number of classes.

### KISS in the small

Redundant conditions are the most common form of accidental complexity:

```
// Before: each condition re-checks what is already known
if (userScore > 100) {
  return "High";
} else if (userScore > 50 && userScore <= 100) {
  return "Medium";
} else if (userScore >= 0 && userScore <= 50) {
  return "Low";
}
return "Invalid";

// After: each condition builds on the previous one
if (userScore > 100) return "High";
if (userScore > 50)  return "Medium";
if (userScore >= 0)  return "Low";
return "Invalid";
```

Before writing a loop, check whether the standard library already does the job:

```
// Before
for (String name : names) {
  if (name.equals(targetName)) {
    return true;
  }
}
return false;

// After
return names.contains(targetName);
```

Finally, complex code does not only slow readers down; it hides bugs. Look at this method:

```
public boolean isEven(int number) {
  if (number % 2 == 0) {
    if (number != 0) {
      return true;
    } else {
      return false;
    }
  } else {
    return false;
  }
}
```

Did you notice that it returns `false` for zero, which *is* an even number? The nested branches make the bug hard to see. The simple version, `return number % 2 == 0;`, has nowhere for that bug to hide.

## Exercises

1. Write the four tests from Step 1 for the original `isActive` method. Then run them against `isActiveOn`. Do they all pass?
2. SubFlow adds a 3-day grace period: a subscription stays active for three days after its end date. Change `isActiveOn`. How many lines did you touch? How many would you have touched in the original version?
3. Find the deepest-nested method in your current project. Can early returns reduce its nesting by at least one level?

### Checklist

☐ Can I explain what this code does in one sentence?

☐ Does any condition re-check something already known?

☐ Can a flag variable be replaced by an early return?

☐ Does the standard library already do this?

☐ Is this solution simple, or simplistic (does it ignore part of the problem)?

## You Aren't Gonna Need It (YAGNI)

### The design problem

Good developers think ahead. That is a strength, until it turns into building features for a future that never arrives. Code written "just in case" costs time to build, slows down the features that *are* needed, and must be read, tested and maintained by everyone, for as long as it exists.

### A problematic example

After the DRY refactoring, a SubFlow developer had an idea: "Today we have one discount, but marketing will surely want coupons, seasonal

promotions and regional prices. Let's build a discount engine now, so we are ready." Two weeks later, the engine was ready:

```
public interface DiscountRule {
  boolean appliesTo(Subscription subscription);
  BigDecimal apply(BigDecimal price);
}

public class DiscountRuleFactory {
  // Reads discounts.json and creates rule objects
  // by class name, using reflection.
  public static List<DiscountRule> load(Path config) {
    ...
  }
}

public class DiscountEngine {
  private final List<DiscountRule> rules;

  public DiscountEngine(List<DiscountRule> rules) {
    this.rules = rules;
  }

  public BigDecimal apply(Subscription subscription,
                          BigDecimal price) {
    BigDecimal result = price;
    for (DiscountRule rule : rules) {
      if (rule.appliesTo(subscription)) {
        result = rule.apply(result);
      }
    }
    return result;
  }
}
```

The engine had a single rule to run: the annual discount, which was already one line in `PricingPolicy`.

Six months later, marketing did ask for a new discount: partner companies get 25% off for *all* their employees' subscriptions. That rule depends on the customer's company, not on the subscription, and it must not be combined with the annual discount. The engine's interface could not express either requirement. It had to be redesigned anyway.

## Symptoms

- Interfaces with exactly one implementation and no plan for a second.

- Configuration options that nobody has ever changed.
- Generic types or plug-in mechanisms used for one concrete case.
- Methods that throw "not implemented" or return placeholder values.
- Code justified with "we might need it" rather than with a ticket or a requirement.

## The principle

YAGNI is one of the practices of Extreme Programming (XP). Ron Jeffries, one of XP's founders, summarized it as: *"Always implement things when you actually need them, never when you just foresee that you need them."*

Martin Fowler explains why speculative features are so expensive. Even if the feature is eventually needed, you pay a **cost of building** it early, a **cost of delay** for the features you did not build in that time, and a **cost of carrying** the extra complexity until then. If you guessed wrong, as with the discount engine, you also pay a **cost of repair**.

## The refactoring

### Step 1: Find the real requirement

Ask: which ticket, customer or regulation needs this code today? For the discount engine, the answer was: one annual discount.

### Step 2: Remove what is not needed

Delete `DiscountRule`, `DiscountRuleFactory`, `DiscountEngine` and `discounts.json`. The annual discount stays exactly where it was, in `PricingPolicy`. Deleted code is the only code with no bugs and no maintenance.

### Step 3: Design for the requirement when it arrives

When the partner discount arrived, the team knew its real shape. They designed for it with real examples in hand. You will see that design in the chapter on SOLID.

### Benefits

- **Faster delivery** of the features customers asked for.
- **Less code** to read, test, secure and upgrade.
- **Better designs** for future features, because they are designed with real requirements instead of guesses.

### Costs and trade-offs

- **YAGNI depends on cheap change.** Postponing a feature is safe only if adding it later is easy. That is why YAGNI works together with DRY, KISS, clean code and tests: they keep the cost of change low.
- **Some rework is expected.** Occasionally you will add a feature later that would have been slightly cheaper to add earlier. That is the price of not building the many features that are never needed.

### When not to apply it

YAGNI is about **features**. It is not an excuse to ignore requirements that are already known, or decisions that are expensive to reverse.

Suppose finance has announced that SubFlow will launch in Europe next quarter, with prices in euros. Building euro invoices, VAT rules and translated emails now would violate YAGNI: the details are not defined yet. But storing every amount as a bare `BigDecimal` in a database column with no currency means that, next quarter, you must migrate every historical invoice. Recording the currency now costs almost nothing:

```
public record Money(BigDecimal amount, Currency currency) { }
```

Apply this test: **what does it cost to add this later, compared with now?** For most code, the cost is about the same, so wait. For data formats, public APIs, security and the ability to observe the system in production, the cost of adding them later is much higher. Make those decisions early and deliberately.

### YAGNI in the small

An interface with one implementation and no second one in sight is speculation:

```
// Avoid this unless you have, or will soon have,
// a second implementation:
public interface OrderServiceInterface {
  void processOrder(int orderId);
}
public class OrderServiceImpl
    implements OrderServiceInterface { ... }

// Start here. Extract the interface when you need it.
public class OrderService {
  public void processOrder(int orderId) { ... }
}
```

The same applies to placeholder methods. A method such as `generateDetailedReport()` that returns `"Not Implemented"` clutters the code and, if it is visible to users, generates a steady stream of "when will this be ready?" questions.

### Exercises

1. Find one interface in your project with exactly one implementation. Is there a concrete reason for it, such as a test double, an external system or a planned second implementation?
2. List three "we might need it" features in your current system. For each, estimate the cost of adding it now versus in a year.
3. SubFlow's product owner says customers *may* want to pause subscriptions someday. Which parts, if any, would you prepare now? Use the cost test above.

### Checklist

☐ Which requirement needs this code today?

☐ Is this abstraction used by more than one real case?

☐ Would adding it later be much more expensive than adding it now?

☐ Is this a feature (wait) or a hard-to-reverse decision (decide now)?

# Loose Coupling and High Cohesion

## The design problem

A change in one part of a system should not break another part that has nothing to do with it. In many real systems it does: a new email provider breaks invoicing, a database migration breaks the user interface, and nobody can run a test without a full environment. These are symptoms of two related problems: classes that know too much about each other, and classes that do too many unrelated things.

**Coupling** is the degree to which one class or module depends on another. **Loose coupling** means modules depend on each other only through small, well-defined contracts, not through their internal details.

**Cohesion** is the degree to which the parts of a class or module belong together. **High cohesion** means everything inside a module serves one clear purpose.

The two ideas work together. Think of a Lego set: each brick is solid inside (high cohesion), and bricks connect through a standard interface that lets you swap one for another (loose coupling).

| Concept | What it encourages | Result |
|---|---|---|
| Loose coupling | Few, explicit dependencies | Modules can be changed or replaced independently |
| High cohesion | A focused internal structure | Modules are easy to understand, name and reuse |

## A problematic example

SubFlow now needs to store each invoice and email it to the customer. The quickest solution was to add everything to `InvoiceService`:

```
public class InvoiceService {
  private final PricingPolicy pricing = new PricingPolicy();

  public void issueInvoice(Subscription subscription,
                           Customer customer)
      throws Exception {
    BigDecimal amount = pricing.priceFor(
        subscription.plan(), subscription.cycle());
```

```
    // store the invoice
    try (Connection db = DriverManager.getConnection(
        "jdbc:mysql://prod-db:3306/subflow",
        "app", "s3cret")) {
      PreparedStatement insert = db.prepareStatement(
          "INSERT INTO invoices (subscription_id, amount)"
          + " VALUES (?, ?)");
      insert.setLong(1, subscription.id());
      insert.setBigDecimal(2, amount);
      insert.executeUpdate();
    }

    // email the customer
    Properties props = new Properties();
    props.put("mail.smtp.host", "smtp.subflow.com");
    Session session = Session.getInstance(props);
    MimeMessage message = new MimeMessage(session);
    message.setRecipients(Message.RecipientType.TO,
        customer.email());
    message.setSubject("Your SubFlow invoice");
    message.setText("Dear " + customer.name()
        + ", your invoice amount is " + amount + ".");
    Transport.send(message);
  }
}
```

## Symptoms

- **Many reasons to change.** This class changes when prices change, when the database schema changes, when the email wording changes and when the company switches email provider. That is low cohesion.
- **Hidden, hard-wired dependencies.** The database address, credentials and mail server are inside the business logic. That is tight coupling, and the password in the source code is also a security problem.
- **No way to test the business logic alone.** Testing the invoice amount requires a running MySQL server and a mail server.
- **Merge conflicts.** Developers working on unrelated features (email templates and database changes) edit the same file.

## The principle

Group together what changes together; separate what changes for different reasons. Let modules communicate through small contracts that hide how the work is done.

Low coupling and high cohesion are not rules you apply once. They are qualities you measure by asking: *when this requirement changes, how many classes must change, and are they all related to the requirement?*

## The refactoring

### Step 1: List the reasons to change

Pricing, storage and notification. Pricing already has a home in `PricingPolicy`. Storage and notification need homes of their own.

### Step 2: Define a small contract for each external system

The invoicing logic needs to *save* an invoice and *send* it. It does not need to know about JDBC or SMTP.

```
public record Invoice(long subscriptionId,
                      String customerEmail,
                      BigDecimal amount) { }

public interface InvoiceRepository {
  void save(Invoice invoice);
}

public interface InvoiceNotifier {
  void send(Invoice invoice);
}
```

### Step 3: Move each detail behind its contract

```
public class JdbcInvoiceRepository
    implements InvoiceRepository {
  private final DataSource dataSource;

  public JdbcInvoiceRepository(DataSource dataSource) {
    this.dataSource = dataSource;   // configured outside
  }

  @Override
  public void save(Invoice invoice) {
    // JDBC code from the original method
  }
}

public class EmailInvoiceNotifier
    implements InvoiceNotifier {
  // SMTP code from the original method
}
```

The connection details now come from configuration, not from the source code.

### Step 4: Let the service coordinate

```
public class InvoiceService {
  private final PricingPolicy pricing;
  private final InvoiceRepository invoices;
  private final InvoiceNotifier notifier;

  public InvoiceService(PricingPolicy pricing,
                        InvoiceRepository invoices,
                        InvoiceNotifier notifier) {
    this.pricing = pricing;
    this.invoices = invoices;
    this.notifier = notifier;
  }

  public Invoice issueInvoice(Subscription subscription,
                              Customer customer) {
    BigDecimal amount = pricing.priceFor(
        subscription.plan(), subscription.cycle());
    Invoice invoice = new Invoice(
        subscription.id(), customer.email(), amount);
    invoices.save(invoice);
    notifier.send(invoice);
    return invoice;
  }
}
```

`InvoiceService` now reads like the business process: calculate, create, save, send. Each collaborator has one job.

Figure 1.2 compares the two designs.

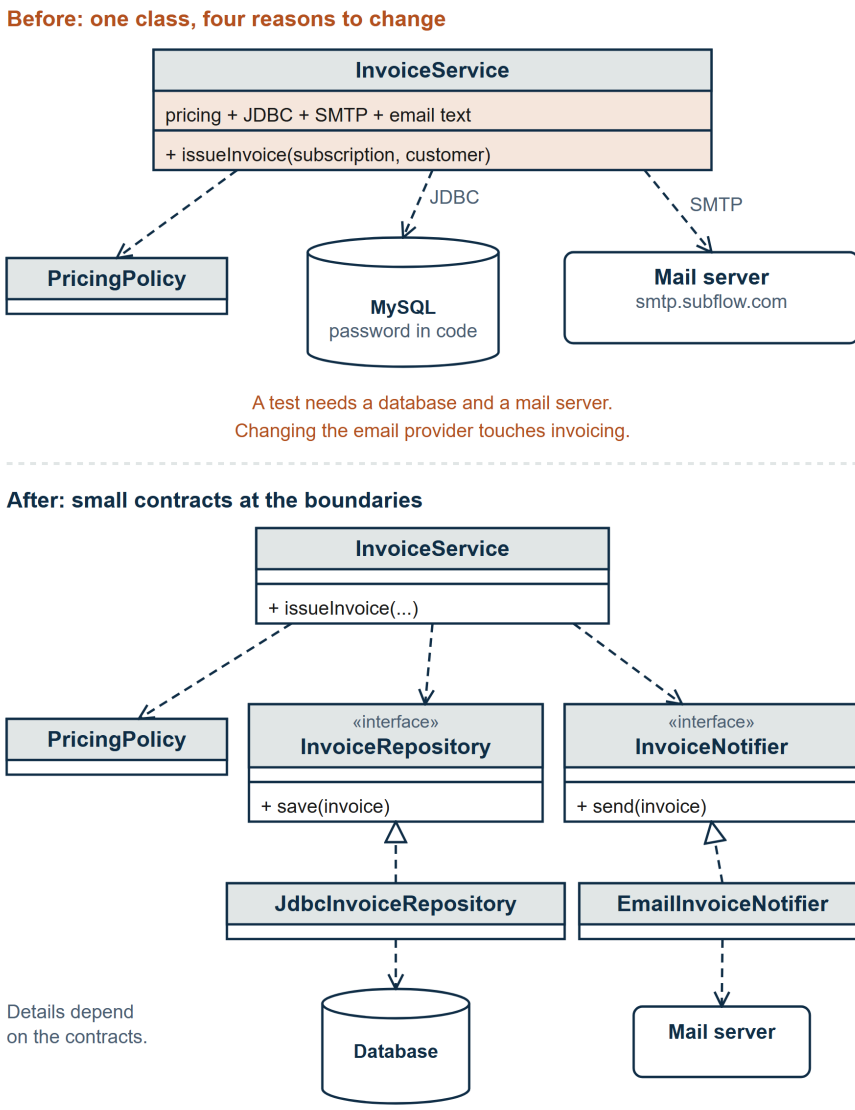



Figure 1.2 – From one class with four reasons to change to small contracts at the boundaries

## Benefits

- **Isolated change.** Switching email provider means writing a new `InvoiceNotifier`. Invoicing does not change.

- **Fast tests without infrastructure.** Because the contracts are small, a test can provide simple in-memory versions, without a mocking library:

```
@Test
void issuesStoresAndSendsTheInvoice() {
  List<Invoice> saved = new ArrayList<>();
  List<Invoice> sent = new ArrayList<>();
  InvoiceService service = new InvoiceService(
      new PricingPolicy(), saved::add, sent::add);

  service.issueInvoice(annualProSubscription(), customer());

  assertEquals(new BigDecimal("203.90"),
               saved.get(0).amount());
  assertEquals(saved, sent);
}
```

- **Parallel work.** One developer can change email templates while another changes the database, without touching the same files.
- **Safer configuration.** Credentials leave the source code.

## Costs and trade-offs

- **More types.** One class became six. Each is easier to understand, but a reader must now navigate between them. Good names and a clear package structure keep this cost low.
- **Interfaces have a cost.** Notice that `PricingPolicy` did *not* get an interface. It is our own business logic, with no external system behind it and only one implementation. The interfaces were added where SubFlow meets the outside world: the database and the email server. That is where loose coupling pays off most.
- **Too much splitting hurts cohesion.** If a single rule is spread across five tiny classes, you have traded one problem for another. Split by reason to change, not by line count.

## When not to apply it

- Small scripts and short-lived prototypes rarely justify contracts and multiple classes.

- Classes that always change together, such as `Plan` and `PricingPolicy`, can be close to each other. Coupling between things that belong together is not a problem; it is cohesion.

### Modular design

The same ideas scale up from classes to **modules**: larger units, such as packages, libraries or services, that group related classes behind a public contract. In Java, the first level of modularity is the package: make classes package-private unless other packages really need them. Since Java 9, the Java Platform Module System lets you go further and declare which packages a module exports.

A well-designed module, like a well-designed class, has high cohesion inside and loose coupling outside. In SubFlow, billing, notifications and customer management will eventually become separate modules, each owning its data and exposing a small contract. You will see this in the chapter on architecture.

### Exercises

1. Write the `JdbcInvoiceRepository.save` method using a `PreparedStatement`.
2. Marketing wants an SMS notification in addition to email. Which classes change? Which do not?
3. Pick a class in your project and list every reason it might change. If there are more than two unrelated reasons, sketch how you would split it.

### Checklist

☐ How many unrelated reasons does this class have to change?

☐ Can I test the business logic without a database, network or file system?

☐ Are configuration and credentials outside the code?

☐ Does each interface sit at a real boundary, or was it added by habit?

☐ Does a typical change touch one module, or many?

# Separation of Concerns

## The design problem

Large problems become manageable when you divide them into smaller ones. This is the old **divide and conquer** strategy, and it applies to software at every level. A **concern** is one part of the problem that can be thought about separately: receiving a web request, applying a business rule, storing data, formatting a response. When concerns are mixed in the same code, you cannot change one without understanding all of them.

## A problematic example

SubFlow customers can cancel their subscription. If they cancel within 14 days of the start date, they get a refund. The first implementation put everything in one method:

```
public String cancel(String requestBody) throws Exception {
  // 1. Read the request
  long id = Long.parseLong(
      requestBody.replaceAll("\\D", ""));

  try (Connection db = dataSource.getConnection()) {
    // 2. Load the data
    ResultSet rs = db.createStatement().executeQuery(
        "SELECT start_date FROM subscriptions WHERE id = "
        + id);
    rs.next();
    LocalDate start = rs.getDate(1).toLocalDate();

    // 3. Apply the business rule
    boolean refund = ChronoUnit.DAYS.between(
        start, LocalDate.now()) <= 14;

    // 4. Save the change
    db.createStatement().executeUpdate(
        "UPDATE subscriptions SET status = 'CANCELED'"
        + " WHERE id = " + id);

    // 5. Build the response
    return "{\"canceled\":true,\"refund\":" + refund + "}";
  }
}
```

## Symptoms

- **One method, five concerns.** HTTP parsing, SQL, a business rule, persistence and JSON formatting.
- **The business rule is buried.** The refund rule, the most important line, is hidden in the middle, and the number 14 repeats knowledge that SubFlow already defines as `REFUND_WINDOW_DAYS`.
- **Untestable rule.** To test the refund window, you need a database, and because the code calls `LocalDate.now()`, the result depends on the day you run the test.
- **Security risk.** Building SQL by concatenating strings opens the door to SQL injection.
- **Fragile input handling.** Removing every non-digit from the request body "works" until a request contains another number.

## The principle

The term *separation of concerns* comes from Edsger Dijkstra, who wrote in 1974 that it was "the only available technique for effective ordering of one's thoughts." In practice: **each part of the code should deal with one concern, and know as little as possible about the others.**

The most common way to separate concerns in business applications is with **layers**. The web layer handles requests and responses. The service layer applies business rules. The data layer stores and retrieves data. Many architecture patterns, such as Model-View-Controller (MVC), are built on this idea.

## The refactoring

### Step 1: Divide the problem

Ask what the method really does, as separate questions: How does the request arrive? What is the business rule? Where is the data? How do we answer? Each answer becomes a separate piece of code.

### Step 2: Give the business rule its own home

```
public class CancellationService {
  static final int REFUND_WINDOW_DAYS = 14;

  private final SubscriptionRepository subscriptions;
  private final Clock clock;

  public CancellationService(
      SubscriptionRepository subscriptions, Clock clock) {
    this.subscriptions = subscriptions;
    this.clock = clock;
  }

  public CancellationResult cancel(long subscriptionId) {
    Subscription subscription = subscriptions
        .findById(subscriptionId)
        .orElseThrow(() ->
            new SubscriptionNotFoundException(subscriptionId));

    LocalDate today = LocalDate.now(clock);
    LocalDate refundDeadline = subscription.startDate()
        .plusDays(REFUND_WINDOW_DAYS);
    boolean refundDue = !today.isAfter(refundDeadline);

    subscriptions.save(subscription.withStatus(
        SubscriptionStatus.CANCELED));
    return new CancellationResult(subscriptionId, refundDue);
  }
}
```

The service knows nothing about HTTP, JSON or SQL. It receives the current date through a `Clock`, so a test can fix the date and check the rule on day 14 and day 15. (`withStatus` is a small method on the `Subscription` record that returns a copy with a new status.)

### Step 3: Move data access behind a repository

```
public interface SubscriptionRepository {
  Optional<Subscription> findById(long id);
  void save(Subscription subscription);
}
```

The JDBC implementation uses `PreparedStatement` parameters instead of string concatenation, which removes the SQL injection risk.

### Step 4: Keep the web layer thin

The web layer becomes a thin translation between HTTP and the service:

```
public record CancellationResult(long subscriptionId,
                                 boolean refundDue) { }

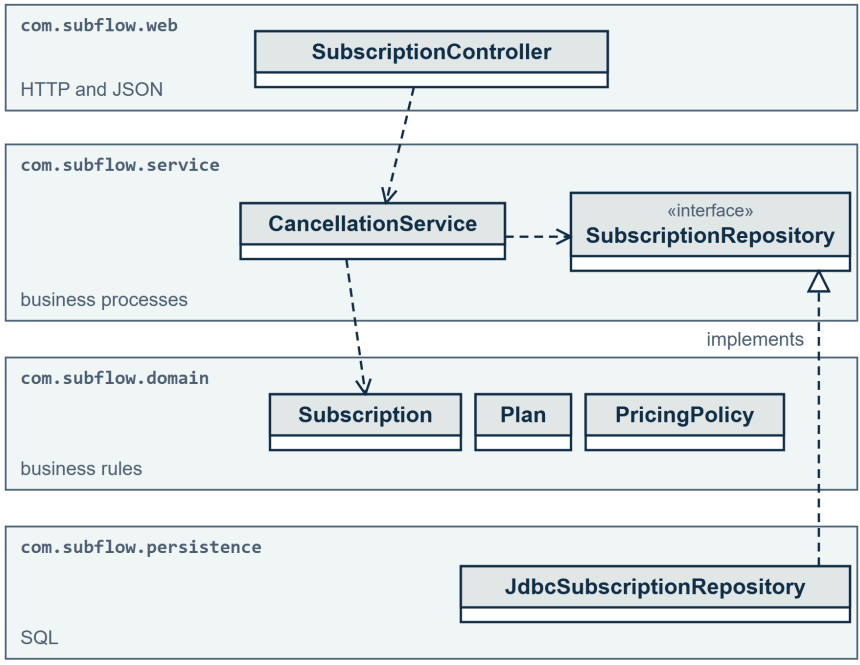
public class SubscriptionController {
  private final CancellationService cancellations;

  public SubscriptionController(
      CancellationService cancellations) {
    this.cancellations = cancellations;
  }

  // Handles: POST /subscriptions/{id}/cancel
  public CancellationResult cancel(long subscriptionId) {
    return cancellations.cancel(subscriptionId);
  }
}
```

Whichever web framework you use, such as Spring Boot or Jakarta EE, its job is to map the URL to this method and convert the result to JSON.

The controller itself contains no business rules and no SQL. Each layer now has one concern:




Each layer has one concern. Dependencies point toward the business rules.

Figure 1.3 – SubFlow organized by layer

## Benefits

- **The business rule is visible and testable** on its own, with a fixed date.
- **Each layer can change independently.** A mobile app can call the same service; the database can change without touching the rule.
- **Safer code.** Input parsing and SQL are handled by tools designed for them.

## Costs and trade-offs

- **More files and more ceremony.** A simple "read one record and show it" feature now passes through three layers.
- **Layers can become pass-through code.** If a service method only calls the repository and returns the result, the layer adds nothing. Accept it for consistency, or let simple reads skip it; decide as a team.
- **Package by layer or by feature?** Figure 1.3 groups code by technical layer. As a system grows, many teams prefer to group by business feature first (`billing/`, `cancellation/`) and by layer inside each feature, so that one change stays in one folder. We will return to this choice in the architecture chapter.

## When not to apply it

- A one-off script or a quick prototype does not need layers.
- Do not separate things that are one concern. Splitting the refund rule across a "validator," a "calculator" and a "decider" class would scatter one idea instead of separating different ones.

## Exercises

1. Write two tests for `CancellationService`: one canceling on day 14 (refund) and one on day 15 (no refund). Use `Clock.fixed(...)`.
2. Implement `withStatus` on the `Subscription` record.
3. Sketch SubFlow's packages organized by feature instead of by layer. Which change in this chapter would touch fewer folders?

## Checklist

☐ Can I point to the one place where each business rule lives?

☐ Does business logic avoid HTTP, JSON and SQL details?

☐ Is the current date or time injected, not read directly?

☐ Is every SQL statement parameterized?

☐ Does each layer add value, or only pass calls through?

# Clean Code

## The design problem

Everybody wants "clean code," but the phrase means little until we make it concrete. In daily work, unclean code shows up as specific costs: an hour lost decoding a variable name, a bug introduced because a function did two things, a comment that described code that had since changed.

In this section, clean code means code that another developer can **read, trust and change** without help from its author. We will look at it through concrete dimensions: **names**, **functions**, **conditionals**, **comments**, **hard-coded values** and **dependencies**. Each move comes from a real project.

## Symptoms

- You need to ask the author what a variable or method means.
- Comments explain *what* the code does, because the code itself does not say it.
- A method's name describes only part of what it does.
- Changing one value requires a search across many files.
- An upgrade or migration is blocked by a library you barely use.

# Names

## Names reveal intent

Someone who reads your code must understand the purpose of a variable, function or class from its name. Here is a real situation:

```
int sId;          // supplier id
int artDelPrice;
```

Refactored:

```
int supplierId;
int articleDeliveredPrice;
```

This matters even more with types from external systems, whose names you cannot change:

```
private Z_E2F_RS_Result e2fResult;  // ingredients recordset
```

Now imagine the comment is missing, and fifty lines later you read:

```
e2f = e2fResult[i];
```

What is `e2f`? You ask the person responsible for this code, but they are not in the office. You send an email, and the reply says they are on holiday for two weeks. A good name would have avoided the whole search:

```
private Z_E2F_RS_Result ingredients;
...
ingredient = ingredients[i];
```

### Names carry context

Different objects often have attributes with the same name. A `Buyer` and a `Supplier` both have an email address. Inside each class, the context is clear, so the field can simply be called `email`. The context gets lost when the value leaves the object:

```
String email = supplier.getEmail();
// fifty lines later: whose email is this?
```

Put the context in the **variable** name, where it is needed:

```
String supplierEmail = supplier.getEmail();
String buyerEmail = buyer.getEmail();
```

Avoid repeating the context inside the class (`supplier.getSupplierEmail()`): it makes every call longer without adding information.

## Functions

### Functions do one thing

In an image-synchronization project, we received image metadata from an external web service as an array of strings. Some values decided

whether the image was valid; others contained candidate image identifiers. The first version did both jobs in one method:

```
private String retrieveImageId(String[] values) {
  if (!values[2].equals("Y") || !values[3].equals("Y"))
    return null;
  String imageId = null;
  // get the first non-null value as the imageId
  if (values[4] != null) {
    imageId = values[4];        // imageAIXId
  } else if (values[5] != null) {
    imageId = values[5];        // imageLIXId
  } else if (values[6] != null) {
    imageId = values[6];        // imageOIXId
  }
  return imageId;
}
```

The name says *retrieve*, but the method also *validates*. A caller who receives `null` cannot tell whether the image was invalid or simply had no identifier. There is a second problem: the numbers 2 to 6 mean something only to someone who has the partner's API documentation open. That documentation said: if positions 2 and 3 both contain `"Y"`, the image is valid, and its identifier is the first non-null value in positions 4 to 6.

Split the two jobs, and record the partner's rules once, in named constants:

```
// Array positions defined in the partner's API documentation
private static final int FIRST_VALIDITY_FLAG = 2;
private static final int SECOND_VALIDITY_FLAG = 3;
private static final int FIRST_IMAGE_ID = 4;
private static final int LAST_IMAGE_ID = 6;
private static final String VALID = "Y";

private boolean isValidImage(String[] values) {
  return VALID.equals(values[FIRST_VALIDITY_FLAG])
      && VALID.equals(values[SECOND_VALIDITY_FLAG]);
}

private Optional<String> firstImageId(String[] values) {
  for (int i = FIRST_IMAGE_ID; i <= LAST_IMAGE_ID; i++) {
    if (values[i] != null) {
      return Optional.of(values[i]);
    }
  }
  return Optional.empty();
}
```

The calling code now reads like the process it implements:

```
public void synchronizeImages() {
  for (Row row : api.getImages().getRows()) {
    String[] values = row.getValues();
    if (isValidImage(values)) {
      firstImageId(values).ifPresent(this::downloadImage);
    }
  }
}
```

### Functions tell a story

Requirements change, and code that started clear slowly stops communicating its flow. This method from a B2B integration sends different kinds of data to a partner called ACME:

```
public void performTask(String process) {
  ACMEWebServiceClient.login();
  if (process.equals("core")) {
    ACMEWebServiceClient.transfer_buyersCoreData_to_ACME();
  }
  if (process.equals("status")) {
    ACMEWebServiceClient
        .transfer_buyersStatusChanges_to_ACME();
  }
  if (process.equals("events")) {
    ACMEWebServiceClient.transfer_events_to_ACME();
  }
  ACMEWebServiceClient.logout();
}
```

The name `performTask` says nothing. The process is an unchecked string. If a transfer fails, the session is never closed. Refactored, the method tells

its story in order: open a session, send one kind of data, always close the session:

```
public enum AcmeExport { BUYER_CORE_DATA,
                         BUYER_STATUS_CHANGES, EVENTS }

public void sendToAcme(AcmeExport export) {
  acmeClient.login();
  try {
    switch (export) {
      case BUYER_CORE_DATA ->
          acmeClient.transferBuyerCoreData();
      case BUYER_STATUS_CHANGES ->
          acmeClient.transferBuyerStatusChanges();
      case EVENTS -> acmeClient.transferEvents();
    }
  } finally {
    acmeClient.logout();
  }
}
```

### Few arguments: use parameter objects

When a function needs many arguments that belong to one concept, the concept is missing from your code:

```
public boolean validateAddress(String street, int number,
    String postalCode, String city, String country) { ... }
```

A Java 17 record names the concept in one line:

```
public record Address(String street, int number,
    String postalCode, String city, String country) { }

public boolean validateAddress(Address address) { ... }
```

The same applies to a function that receives an untyped array and decodes each position:

```
public ListOrdersResponse listOrders(String[] values) {
  Integer buyerId = Integer.valueOf(values[0]);
  Integer supplierId = Integer.valueOf(values[1]);
  LocalDate orderDateFrom = LocalDate.parse(values[2]);
  // ... five more positions
}
```

Replacing the array with eight separate parameters would trade one problem for another. A parameter object gives each value a name and a type, and gives the search criteria a home for future validation:

```
public record OrderSearch(
    int buyerId,
    int supplierId,
    LocalDate orderDateFrom,
    LocalDate orderDateTo,
    String sortBy,
    SortOrder sortOrder,
    int offset,
    int limit) { }

public ListOrdersResponse listOrders(OrderSearch search) {
  ...
}
```

### Overloading to extend without breaking callers

Suppose a function that subscribes buyers to an external service is called from many places:

```
subscriberId = wsClient.subscribe(email, buyer);
```

A new requirement asks some callers to send a tag identifier as well. Adding a third parameter would force every existing caller to pass `null`. Java does not have default parameter values, but an overload achieves the same result:

```
public int subscribe(String email, Buyer buyer,
                     int tagId) { ... }

public int subscribe(String email, Buyer buyer) {
  ...   // existing behavior, unchanged
}
```

Only the new callers use the new version. Use this technique with care: more than two or three overloads of the same method become hard to tell apart, and a parameter object is usually a better choice.

## Conditionals

### Choose clarity over compactness

To build an article's image URL, we must check that the image is valid and, if it is restricted, that the partner may see it. A conditional operator fits it on one line:

```
String articleImageURL =
    (imageId <= 0 || (imageIsRestricted && !partnerCanSeeImage))
        ? null : IMAGE_URL + imageId;
```

It is compact, but each new condition makes it harder to read and to change. Extract a method with **guard clauses**, one per reason to reject:

```
private String articleImageUrl(int imageId,
    boolean imageIsRestricted, boolean partnerCanSeeImage) {
  if (imageId <= 0) {
    return null;
  }
  if (imageIsRestricted && !partnerCanSeeImage) {
    return null;
  }
  return IMAGE_URL + imageId;
}
```

Each rule now has its own line, and a new rule is a new guard clause.

## Comments

### Don't comment bad code; rewrite it

An external API sometimes returns more articles than we requested. Before processing them, we skip the extra ones, and a comment warns colleagues:

```
// only requested articles
if (!mapOfArticles.containsKey(articleId))
  continue;
```

A better variable name makes the comment unnecessary:

```
if (!requestedArticles.containsKey(articleId))
  continue;
```

### Comment on what the code cannot say

Good names and small functions explain *what* the code does. They cannot explain *why* a decision was made, especially when the reason is

outside the code. In one project, a CSV file exported manually from a database sometimes arrived with the wrong character encoding, and the Java client produced corrupted article names. The explanation existed only in an informal email. That is exactly what a comment is for, next to code that makes the decision explicit:

```
// The CSV is exported manually from the ERP database.
// The export must be saved as UTF-8; other encodings
// corrupt accented characters in article names.
List<String> lines = Files.readAllLines(
    file, StandardCharsets.UTF_8);
```

## Hard-coded values

### Isolate values that will change

Hard coding means writing data directly into the source code instead of reading it from an external source. Sometimes it is unavoidable in the short term. In one company, a customer could only see images from certain providers, because of image copyright problems. The proper solution was a database function, which required the database team, new parameter classes and a full test cycle. With no resources available, the team shipped this:

```
boolean pictureIsRestricted =
    result.getProviderId() == "530636"
        || result.getProviderId() == "36507";
```

Besides hard coding the providers, this code has a classic Java bug: `==` compares String *references*, not their contents, so it can return `false` even when the identifiers match. Strings must be compared with `equals()`.

The bigger problem appears later. The same list is needed in other modules, more providers are added, and each copy is edited by hand.

Even when you cannot build the proper solution yet, you can isolate the decision behind a small contract:

```
public interface RestrictedProviders {
  boolean contains(String providerId);
}

public class FileRestrictedProviders
    implements RestrictedProviders {
  private final Set<String> providerIds;

  public FileRestrictedProviders(Path file)
      throws IOException {
    this.providerIds = Files.readAllLines(file).stream()
        .map(String::strip)
        .filter(line -> !line.isEmpty())
        .collect(Collectors.toUnmodifiableSet());
  }

  @Override
  public boolean contains(String providerId) {
    return providerIds.contains(providerId);
  }
}
```

Every module uses the same check:

```
boolean pictureIsRestricted =
    restrictedProviders.contains(result.getProviderId());
```

Notice that if the file cannot be read, the constructor fails and the application refuses to start. Failing silently with an empty list would be worse: every image would be treated as unrestricted, which is exactly the copyright problem we are trying to prevent.

When the database function is finally delivered, you write a second implementation, `DatabaseRestrictedProviders`, and change one line of configuration. No caller changes.

## Dependencies

### Depend only on what you use

We were migrating an application to a new application server and discovered that a library our code depended on was no longer supported:

```
import org.apache.axis.transport.http.HTTPConstants;

HttpServletRequest request = (HttpServletRequest)
    context.get(HTTPConstants.MC_HTTP_SERVLETREQUEST);
```

We spent a lot of effort looking for a compatible version of the Axis library. Then we realized that our code used only one thing from it: this constant, whose value is the string `"transport.http.servletRequest"`. The solution was to declare the constant in our own code:

```
public static final String MC_HTTP_SERVLETREQUEST =
    "transport.http.servletRequest";

HttpServletRequest request = (HttpServletRequest)
    context.get(MC_HTTP_SERVLETREQUEST);
```

And we removed the old library from the project. Every dependency is a commitment: it must be upgraded, secured and kept compatible. Before adding one, ask how much of it you will actually use.

### Costs and trade-offs

- **Readability is partly a team decision.** Whether a stream pipeline or a loop is clearer, or where braces go, depends on your team's experience. Agree on conventions and let a formatter enforce the mechanical ones.
- **Refactoring takes time.** Clean up the code you are already changing ("leave it better than you found it") rather than stopping a project to clean everything.
- **Rules of thumb are not rules.** "Functions should be short" is useful guidance, but a 25-line function that reads top to bottom can be clearer than five 5-line functions you must jump between.

## When not to apply it

- Do not rewrite stable code that nobody needs to change just to make it prettier. Every change carries risk.
- Do not mix large clean-up changes with feature changes in the same commit. Reviewers cannot see the feature through the noise.

## Exercises

1. Find three abbreviated names in your project and rename them. Did any rename reveal a misunderstanding about what the variable holds?
2. Find a function whose name describes only part of what it does. Split it or rename it.
3. Find one comment that explains *what* the code does. Can you delete it by improving a name?
4. List your project's dependencies. For one of them, count how many of its classes your code actually uses.

## Checklist

☐ Can a new team member understand each name without asking?

☐ Does each function do what its name says, and only that?

☐ Are related arguments grouped into a parameter object?

☐ Do comments explain *why*, not *what*?

☐ Are values that will change isolated in one place?

☐ Does the code fail loudly when something essential is missing?

☐ Is every dependency worth what it costs?

## Chapter Summary

Each principle in this chapter solves a specific problem, and each one causes a different problem when it is overused. Keep this table nearby during code reviews:

| Principle | Problem it solves | Risk when overused |
|---|---|---|
| DRY | Business knowledge copied in several places drifts apart | Merging coincidentally similar code; the wrong abstraction |
| KISS | Accidental complexity hides bugs and slows readers down | Simplistic solutions that ignore real complexity |
| YAGNI | Speculative features cost time and complexity | Ignoring known requirements and hard-to-reverse decisions |
| Loose coupling / high cohesion | Changes ripple through unrelated code | Interfaces everywhere; logic scattered across tiny classes |
| Separation of concerns | Mixed responsibilities make code untestable and fragile | Layers that only pass calls through |
| Clean code | Code that only its author understands | Endless polishing of stable code; rules applied without judgment |

### SubFlow so far

At the end of this chapter, SubFlow has a clear shape:

- **Domain**: `Plan`, `BillingCycle`, `SubscriptionStatus`, `Subscription`, `Invoice`, and `PricingPolicy`, the single home of the pricing rule.
- **Services**: `InvoiceService` and `CancellationService`, which coordinate the business processes.
- **Boundaries**: `InvoiceRepository`, `SubscriptionRepository` and `InvoiceNotifier`, small contracts between SubFlow and the database and email server.
- **Web**: a thin `SubscriptionController`.

The design is good enough for today, and that is the point. In the next chapter, you will look inside these classes and see how object-oriented concepts, such as encapsulation, inheritance, polymorphism and composition, help SubFlow protect its rules as new requirements arrive.

The first one is already waiting: partner companies want their own prices.

END OF THE SAMPLE

# SubFlow is just getting started

Chapter 1 gave SubFlow a clear shape. The rest of the book keeps growing the same system as real requirements arrive: partner prices, promotions, a second payment provider, retries, outages and a business that says no.


PART I · CHANGEABLE CODE

2 Object-Oriented Design: Protecting Your Rules
3 SOLID Principles
4 Design for Testability
5 When Principles Conflict

PART II · CHANGEABLE SYSTEMS

6 Designing Boundaries: Domain, Data and APIs
7 From Requirements to Architecture
8 Communicating Design: UML and Beyond

PART III · DESIGN IN PRACTICE

9 Case Studies: Three Systems from a Blank Page
10 Soft Skills for Design Decisions
11 Conclusions: Growing as a Designer

APPENDICES

Checklists, symptom-to-principle guide, glossary, reading


## Read the whole book

Paperback, hardcover and Kindle editions on Amazon.

| | |
|---|---|
| On Amazon | **link.amazon/B06Wgb9GT** |
| Companion code | **github.com/mgamio/subFlow** |
| More articles | **codersite.dev** |

Copyright © 2022, 2026 Moises Gamio. Sample chapter, free to share unchanged.